Semantic cache for LLM serving

Name: Pojesh Kumar R
Reg: 22BAI1373

Guide: Dr. Kumar R - 50304

Approved

Semantic cache for LLM serving

Name: Pojesh Kumar R
Reg: 22BAI1373

Guide: Dr. Kumar R - 50304

CONTENTS

- Introduction and Objective
- System Architecture
- System Design
- Implementation Details
- Results and Analysis
- Challenges & Solutions
- Remaining Work
- References

INTRODUCTION

The Problem :

- **High LLM inference costs**: Production-scale deployments face significant API costs and latency with large query volumes.

- **Query redundancy**: Research analyzing 27K ChatGPT conversations shows ~31% of queries are semantically similar to previous ones — but existing caches capture only a fraction.

- **Misclassification**: Fixed-threshold similarity methods often misclassify queries and fail to adapt across diverse domains.

- **No per-entry adaptation**: All existing systems apply a single global threshold — a query about "capital of France" (needs high precision) gets the same threshold as "explain machine learning" (tolerates more variation).

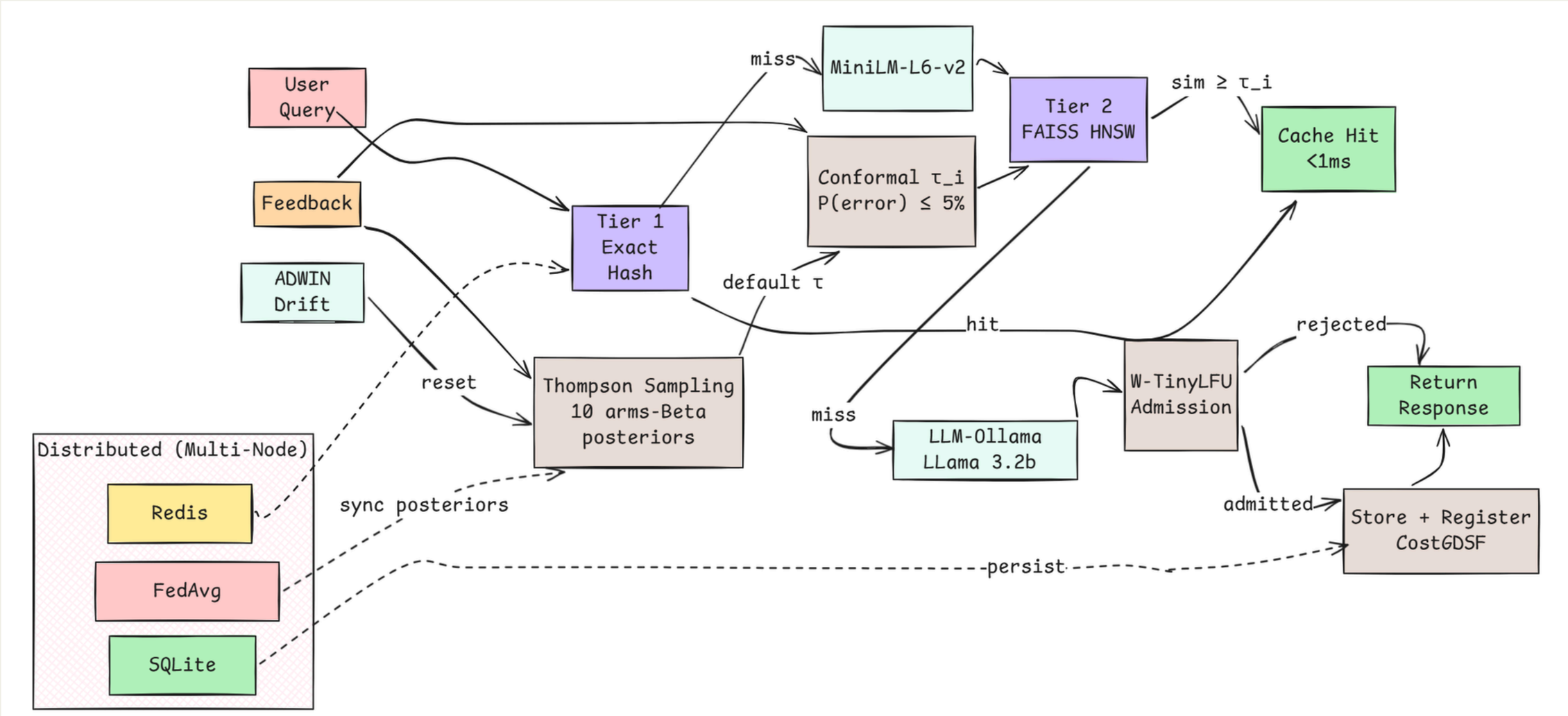
Our Solution: Semantic Cache

A fault-tolerant adaptive semantic cache that learns per-entry similarity thresholds with formal correctness guarantees, using Thompson Sampling bandits for exploration and conformal prediction for calibration.

OBJECTIVE

- **Adaptive thresholds:** Replace fixed global thresholds with per-entry conformal thresholds that learn from feedback
- **Intelligent admission & eviction:** Implement TinyLFU frequency-gated admission and CostGDSF cost-aware eviction to maximize cache utility
- **Multi-turn context awareness:** Support session-based query context for conversational workloads
- **Thompson Sampling exploration:** Use multi-armed bandit with Beta posteriors and ADWIN drift detection to learn optimal thresholds
- **Federated parameter sync:** Enable distributed deployment with FedAvg-based bandit posterior synchronization across shards
- **Comprehensive benchmarking:** Compare against baseline strategies(GPTCache, MeanCache, vCache) on Natural Questions dataset

SYSTEM ARCHITECTURE



SYSTEM DESIGN - QUERY PIPELINE

Stage	Component	Complexity	Latency
Tier 1	Exact Hash Lookup (SHA-256)	O(1)	<1ms
Tier 2	FAISS HNSW Semantic Search	O(log n)	2-15ms
Embedding	paraphrase-MiniLM-L6-v2	O(d)	5-20ms
Threshold	Per-entry conformal τ_i	O(k log k)	<0.1ms
Miss	LLM Provider (Ollama/Groq)	Network	500-6000ms

SYSTEM DESIGN - CACHE MANAGEMENT

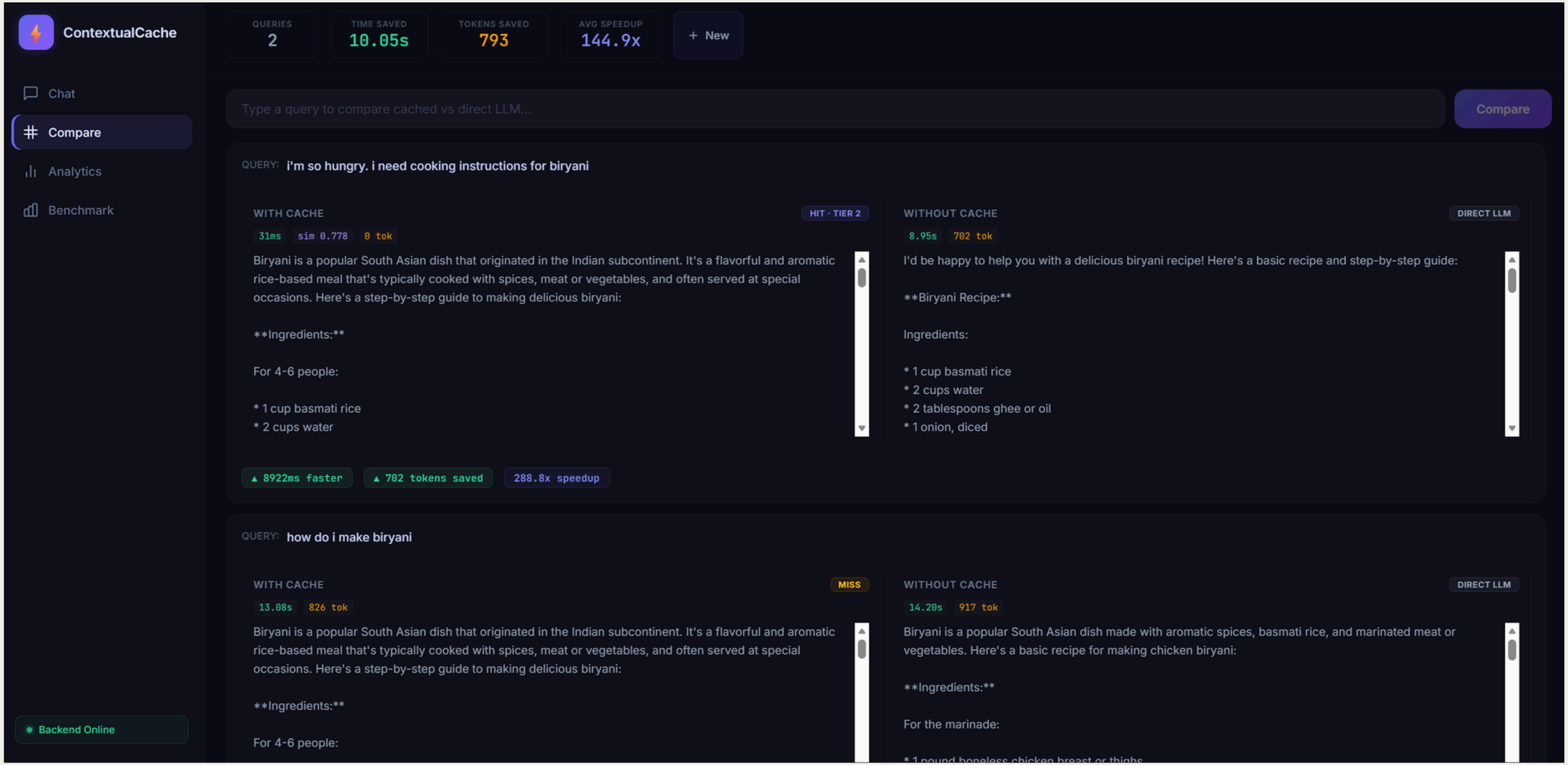
Component	Algorithm	Purpose
Admission	Semantic-W-TinyLFU	Reject one-hit-wonders and allow based on frequency
Eviction	CostGDSF	Priority = freq × regen_cost / storage + L (keeps expensive entries)
Threshold Learning	Thompson Sampling (10 arms, Beta posteriors)	Learn optimal default τ for new entries
Drift Detection	ADWIN	Reset bandit on distribution shift
Session Context	EMA fusion (α=0.85)	Multi-turn conversational awareness
Fault Isolation	Circuit Breaker	Fault isolation

THOMPSON SAMPLING- ALGORITHM

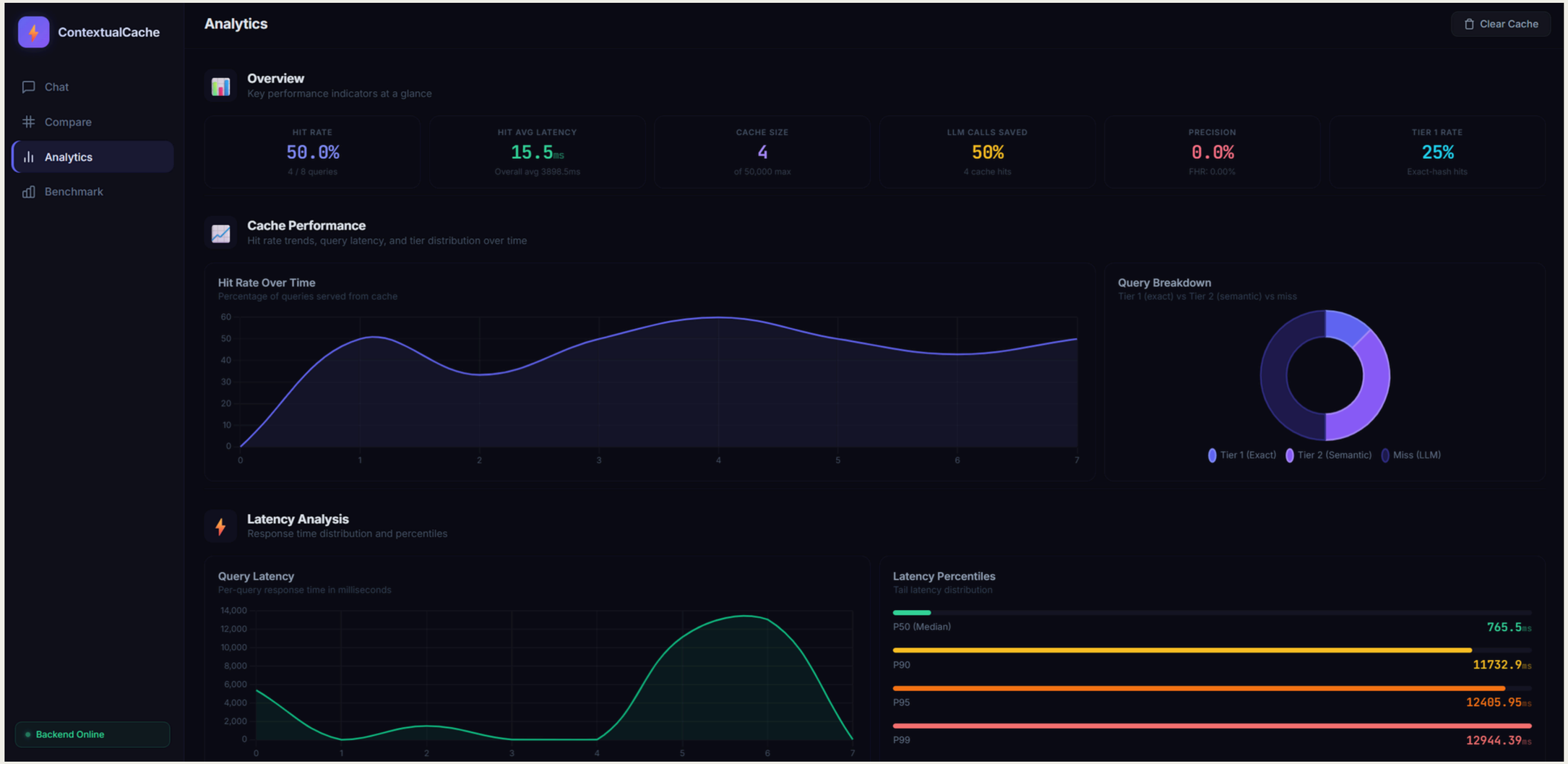
```
10 arms: τ ∈ [0.65, 0.683, 0.717, 0.75, 0.783, 0.817, 0.85, 0.883, 0.917, 0.95]
For each arm i:
    posterior ~ Beta(α_i, β_i)      # initially Beta(1,1) = uniform

On query:
    sample s_i ~ Beta(α_i, β_i) for each arm
    select arm* = argmax(s_i)        # Thompson Sampling
    use τ = threshold_arms[arm*]

On feedback:
    correct hit  → α[arm*] += 1     # arm looks better
    incorrect hit → β[arm*] += 1    # arm looks worse
    uncertain    → skip              # no noise injection

Drift detection (ADWIN):
    if distribution shift detected → reset all α, β to (2, 2)
```

Key insight: The bandit explores the threshold space while conformal prediction provides per-entry fine-tuning. New entries start with the bandit's best guess; after 10+ feedback signals, conformal takes over with formal guarantees.

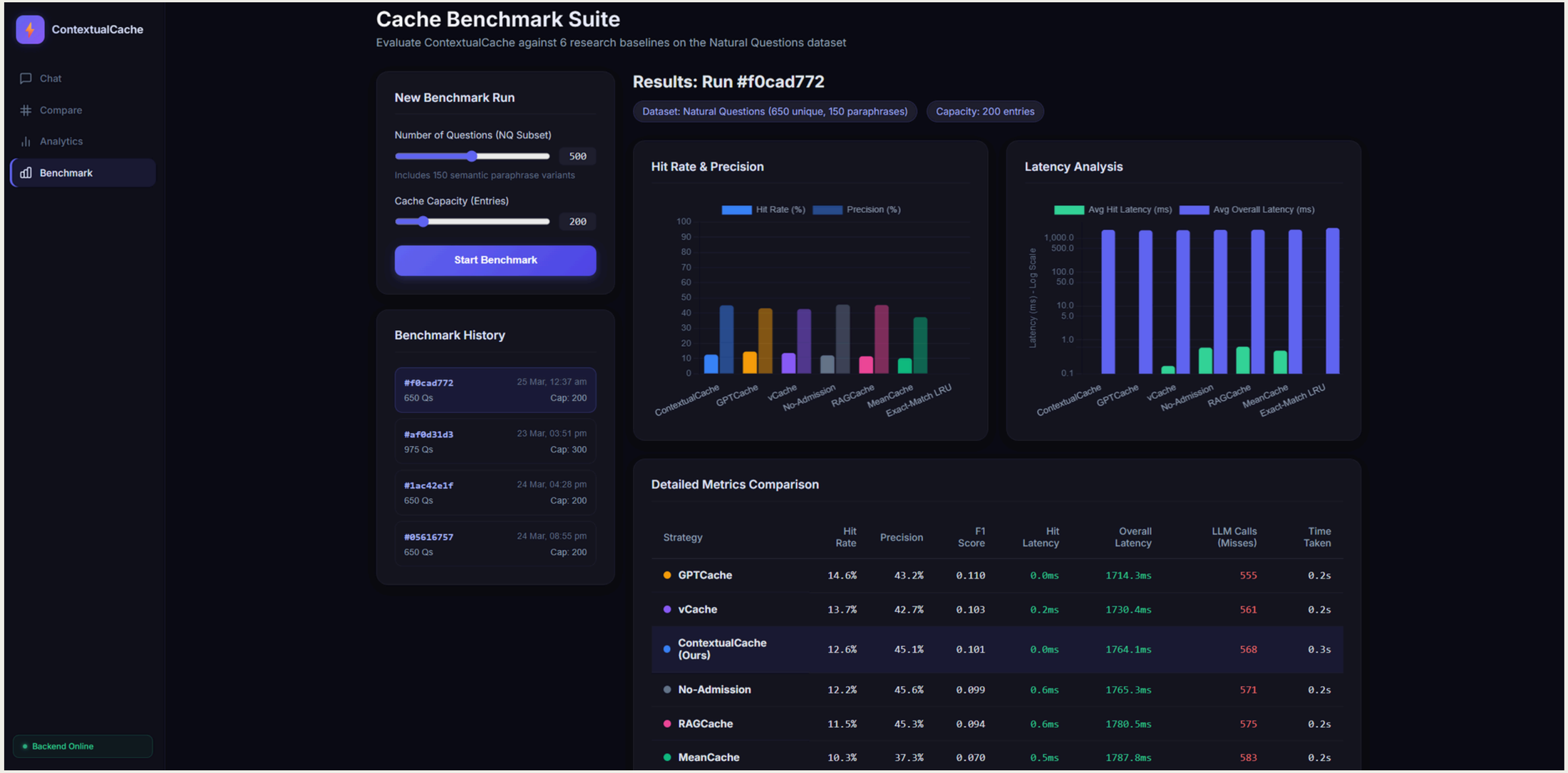
IMPLEMENTATION - CACHE HIT



IMPLEMENTATION - ANALYTICS DASHBOARD



IMPLEMENTATION - BENCHMARK



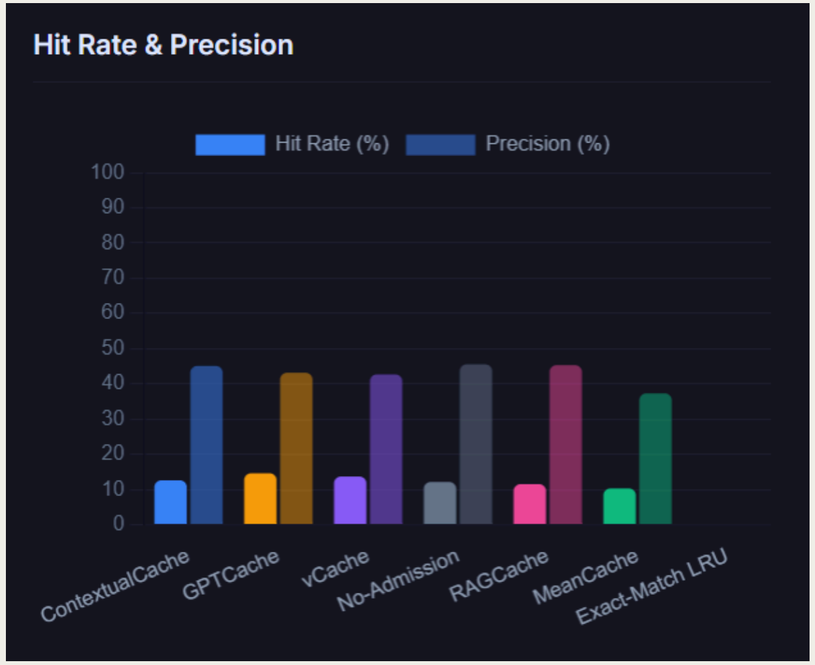
RESULTS - BENCHMARK OVERVIEW

Strategy	Hit Rate	Precision	F1	Correct Hits	Incorrect Hits	LLM Calls
GPTCache	14.62%	43.16%	0.110	41	54	555
vCache	13.69%	42.70%	0.103	38	51	561
SemanticCache (Ours)	12.62%	**45.12%**	0.101	37	45	568
No-Admission	12.15%	45.57%	0.099	36	43	571
RAGCache	11.54%	45.33%	0.094	34	41	575
MeanCache	10.31%	37.31%	0.070	25	42	583
Exact-Match LRU	0.00%	—	—	0	0	650

Dataset: Natural Questions (NQ) - 650 queries (500 unique + 150 paraphrases)
Cache Capacity: 200 entries

RESULTS - PRECISION ANALYSIS

Semantic Cache Achieves Highest Precision Among High-Hit-Rate Strategies



Metric	GPTCache	vCache	Ours
Total Hits	95	89	82
Correct Hits	41	38	37
Incorrect Hits	54	51	45
Precision	43.16%	42.70%	**45.12%**

- GPTCache serves 54 wrong answers out of 95 hits
- ContextualCache serves 45 wrong answers out of 82 hits
- 9 fewer false positives-per-entry conformal thresholds reject borderline matches that other systems accept
- This is the intended design: higher precision = fewer users see incorrect cached responses

CHALLENGES AND SOLUTIONS

Challenge	Solution
Conformal thresholds activation	Added correctness feedback loop using NQ gold answers for conformal thresholds and bandit calibration
LLM had no conversation context	Built chat history from SQLite persistence, pass to LLM via Ollama
Embedding model suboptimal (all-MiniLM-L6-v2 = general purpose)	Switched to paraphrase-MiniLM-L6-v2 (trained on paraphrase data, better hit rate)
FAISS memory leak	Wrapped FAISS in explicit ID management) in avoid stale vectors

REMAINING WORK

- **Multi-node deployment testing:** Gossip protocol components for fed are implemented but need integration testing across nodes
- **Embedding model upgrades**: E5-base-v2, instructor models for domain-specific caching
- **Additional Benchmarking:** QQP for comprehensive evaluation
- **Production load testing:** Stress testing with concurrent users via wrk/locust

REFERENCES

1. *K. Haqiq, "MinCache: A hybrid cache system for efficient chatbots with hierarchical embedding matching and LLM," Proc. Future Gener. Comput. Syst., vol. 165, pp. 107-120, 2025.*
2. *W. Gill, A. Elidrisi, et al., "MeanCache: User-Centric Semantic Caching for LLM Web Services," arXiv:2403.02694, 2024.*
3. *S. Regmi and C. P. Pun, "GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching," arXiv:2411.05276, 2024.*
4. *A. Schroeder, et al., "vCache: Per-Entry Conformal Thresholds for Semantic Caching," UC Berkeley, 2025.*
5. *H. Li, et al., "Context-based Semantic Caching for LLM Applications," Proc. IEEE ICWS, 2024.*
6. *Z. Li, et al., "SCALM: Towards Semantic Caching for Automated Chat Services with LLMs," Proc. IEEE/ACM IWQoS, 2024.*
7. *G. Einziger, R. Friedman, B. Manes, "TinyLFU: A Highly Efficient Cache Admission Policy," ACM Trans. Storage, vol. 13, no. 4, 2017.*
8. *Y. Jin, et al., "RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation," arXiv:2404.12457, 2024.*
9. *B. McMahan, et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," Proc. AISTATS, 2017.*
10. *A. Bifet and R. Gavalda, "Learning from Time-Changing Data with Adaptive Windowing," Proc. SDM, 2007.*
11. *V. Vovk, A. Gammerman, G. Shafer, "Algorithmic Learning in a Random World," Springer, 2005.*
12. *M. Young, "The GDSF Online Caching Algorithm," Tech Report, 2000.*
13. *X. Gao, et al., "Online Context Caching for Distributed LLM Serving," Proc. IEEE IPDPS, 2025.*
14. *F. Zhu, et al., "Semantic Caching for Large Language Model Cost Reduction," Proc. IEEE GLOBECOM, 2023.*
15. *R. Zha, et al., "Online Optimization for Semantic Cache Hit Prediction," Proc. IEEE Big Data, pp. 690-699, 2023.*

Thank you